

Project 6-2 Submission

Vector Pseudocode

START PROGRAM

CREATE Course structure

 courseNumber

 courseName

 prerequisites

CREATE vector called courseList

FUNCTION LoadCourses(fileName)

 OPEN file

 IF file does not open

 PRINT error message

 RETURN

 FOR each line in file

 SPLIT line by commas

 IF line has fewer than 2 items

 PRINT format error

 CONTINUE

 CREATE new Course

 STORE first item as courseNumber

 STORE second item as courseName

 STORE remaining items as prerequisites

 ADD Course to courseList

 END FOR

 FOR each course in courseList

 FOR each prerequisite

 IF prerequisite is not found as a course number in courseList

 PRINT prerequisite error

 END IF

 END FOR

 END FOR

 CLOSE file

END FUNCTION

FUNCTION PrintAllCourses()

 SORT courseList by courseNumber in alphanumeric order

 FOR each course in courseList

 PRINT courseNumber and courseName

 END FOR

END FUNCTION

FUNCTION SearchCourse(courseNumber)

 FOR each course in courseList

 IF courseNumber matches

 PRINT courseNumber and courseName

 IF course has no prerequisites

 PRINT "Prerequisites: None"

```

ELSE
    PRINT "Prerequisites:"

    FOR each prerequisite in course prerequisites

        FOR each prereqCourse in courseList

            IF prerequisite matches prereqCourse courseNumber
                PRINT prereqCourse courseNumber and prereqCourse courseName
            END IF

        END FOR

    END FOR

END IF

RETURN

END IF

END FOR

PRINT course not found

END FUNCTION

MAIN

DISPLAY menu

    1. Load Course Data
    2. Print Course List
    3. Print Course Information
    9. Exit

WHILE user does not choose Exit

    GET user choice

    IF choice is Load Course Data
        CALL LoadCourses

    IF choice is Print Course List
        CALL PrintAllCourses

```

```
IF choice is Print Course Information
  GET course number
  CALL SearchCourse
```

```
END WHILE
```

```
END PROGRAM
```

Hash Table Pseudocode

```
START PROGRAM
```

```
CREATE Course structure
  courseNumber
  courseTitle
  prerequisites
```

```
CREATE hash table called courseTable
```

```
DISPLAY menu
```

1. Load Course Data
2. Print Course List
3. Print Course Information
9. Exit

```
FUNCTION LoadCourses(fileName)
```

```
  OPEN CSV file
```

```
  IF file does not open
    PRINT error message
    RETURN
```

```
  FOR each line in file
```

```
    SPLIT line by commas
```

```
    IF line has fewer than 2 items
      PRINT format error
      CONTINUE
```

```
    CREATE Course object
```

```
    STORE first item as courseNumber
    STORE second item as courseTitle
```

STORE remaining items as prerequisites

INSERT Course into hash table using courseNumber as key

END FOR

CLOSE file

END FUNCTION

FUNCTION Hash(courseNumber)

CONVERT courseNumber into a numeric value

DIVIDE numeric value by table size

RETURN remainder as hash key

END FUNCTION

FUNCTION InsertCourse(course)

CREATE hash key from courseNumber

IF bucket is empty

STORE Course in bucket

ELSE

MOVE through linked list

ADD Course to end of list

END FUNCTION

FUNCTION PrintCourseList()

CREATE temporary list

LOOP through each bucket in hash table

ADD each Course to temporary list

ADD linked Courses to temporary list

END LOOP

SORT temporary list by courseNumber

```
    FOR each Course in temporary list
        PRINT courseNumber and courseTitle
    END FOR
```

```
END FUNCTION
```

```
FUNCTION SearchCourse()
```

```
    ASK user for courseNumber
```

```
    CREATE hash key from courseNumber
```

```
    CHECK bucket at that key
```

```
    MOVE through linked list if needed
```

```
    IF courseNumber matches
```

```
        PRINT courseNumber and courseTitle
```

```
        IF course has no prerequisites
```

```
            PRINT "Prerequisites: None"
```

```
        ELSE
```

```
            PRINT "Prerequisites:"
```

```
            FOR each prerequisite in course prerequisites
```

```
                SEARCH hash table using prerequisite courseNumber
```

```
                IF prerequisite course is found
```

```
                    PRINT prerequisite courseNumber and prerequisite courseTitle
```

```
                ELSE
```

```
                    PRINT prerequisite not found
```

```
                END IF
```

```
            END FOR
```

```
        END IF
```

```
    RETURN
```

```
END IF
```

PRINT course not found

END FUNCTION

END PROGRAM

Binary Search Tree Pseudocode

START PROGRAM

CREATE Course structure

courseNumber

courseTitle

prerequisites

CREATE empty binary search tree

DISPLAY menu

1. Load Course Data
2. Print Course List
3. Print Course Information
9. Exit

FUNCTION LoadCourses(fileName)

OPEN CSV file

IF file does not open

PRINT error message

RETURN

FOR each record in file

SPLIT record by commas

IF record has fewer than 2 items

PRINT format error

CONTINUE

CREATE Course object

STORE first item as courseNumber

STORE second item as courseTitle

STORE remaining items as prerequisites

```

    INSERT Course into binary search tree

END FOR

END FUNCTION

FUNCTION InsertCourse()

    IF tree is empty
        CREATE root node

    ELSE

        COMPARE courseNumber with current node

        IF smaller
            MOVE left

        IF larger
            MOVE right

        INSERT Course in correct position

    END FUNCTION

FUNCTION SearchCourse()

    START at root node

    WHILE current node exists

        COMPARE courseNumber with current node

        IF match found

            PRINT courseNumber and courseTitle

            IF course has no prerequisites
                PRINT "Prerequisites: None"

            ELSE

                PRINT "Prerequisites:"

                FOR each prerequisite in course prerequisites

```

```

        SEARCH tree using prerequisite courseNumber

        IF prerequisite course is found
            PRINT prerequisite courseNumber and prerequisite courseTitle
        ELSE
            PRINT prerequisite not found
        END IF

    END FOR

END IF

RETURN

END IF

IF courseNumber is smaller
    MOVE left

IF courseNumber is larger
    MOVE right

END WHILE

PRINT course not found

END FUNCTION

FUNCTION DisplayCourseList()

    VISIT left subtree

    DISPLAY current courseNumber and courseTitle

    VISIT right subtree

END FUNCTION

END PROGRAM

```

Runtime Analysis

Big O Analysis Table

Data Structure	Loading File and Creating Course Objects	Print Sorted Course List	Search for Course	Memory Usage
Vector	$O(n)$	$O(n \log n)$	$O(n)$	$O(n)$
Hash Table	$O(n)$ average	$O(n \log n)$	$O(1)$ average	$O(n)$
Binary Search Tree	$O(n \log n)$ average	$O(n)$	$O(\log n)$ average	$O(n)$

Cost Analysis

Vector

Operation	Cost Per Execution	Number of Executions	Total Cost
Read line from file	1	n	n
Parse line	1	n	n
Create course object	1	n	n
Add course to vector	1	n	n

Total Running Time: $O(n)$

Hash Table

Operation	Cost Per Execution	Number of Executions	Total Cost
Read line from file	1	n	n
Parse line	1	n	n
Create course object	1	n	n
Insert into hash table	1 average	n	n

Total Running Time: $O(n)$

Binary Search Tree

Operation	Cost Per Execution	Number of Executions	Total Cost
------------------	---------------------------	---------------------------------	-------------------

Read line from file	1	n	n
Parse line	1	n	n
Create course object	1	n	n
Insert into tree	$O(\log n)$ average	n	$O(n \log n)$

Total Running Time: $O(n \log n)$

Advantages and Disadvantages

Vector

Advantages

- Simple to implement and understand.
- Uses contiguous memory which allows efficient traversal.
- Works well for smaller datasets.
- Easy to sort using standard sorting algorithms.

Disadvantages

- Searching for a specific course requires a linear search.
- Performance decreases as the number of courses grows.
- The entire vector must be sorted before printing courses in order.

Hash Table

Advantages

- Very fast course lookups using the course number as a key.
- Average search time is $O(1)$.
- Efficient for finding individual course information.

Disadvantages

- Courses are not stored in sorted order.
- Additional processing is required to create a sorted list for printing.
- Performance can decrease when collisions occur.

Binary Search Tree

Advantages

- Stores courses in sorted order based on course number.
- Efficient searching with $O(\log n)$ average performance.
- In-order traversal naturally prints courses in alphanumeric order.
- Supports both major advisor requirements effectively.

Disadvantages

- More complex to implement than a vector.
- Performance can degrade if the tree becomes unbalanced.
- Insertion is slower than a hash table on average.

Recommendation

After evaluating the vector, hash table, and binary search tree data structures, I recommend using a binary search tree for the ABCU advising program.

Although vectors and hash tables both provide advantages, the binary search tree best supports the program requirements. The advisors need to print all courses in alphanumeric order and search for specific course information, including prerequisites. A binary search tree naturally stores courses in sorted order by course number, allowing an in-order traversal to print the entire course list without requiring an additional sorting operation.

While a hash table provides the fastest average search time, it does not maintain courses in sorted order. As a result, extra processing is required to create and sort a temporary list before displaying the course catalog. A vector is easy to implement, but searching for individual courses requires examining each course until a match is found, resulting in slower performance as the number of courses increases.

Based on the Big O analysis and the project requirements, the binary search tree provides the best balance between efficient searching and ordered output. Therefore, it is the most appropriate data structure to use when developing the final ABCU advising application.